Project plan and research evidence roadmap

An Agentic Framework for Automatic Project-Specific NLP Pipeline Design and Domain-Adaptive Language Understanding

Prepared: 11 October 2026, Asia/Kolkata. **Version: 1.2** (v1.1 corrected after an independent review, the same day). Status: planning complete; implementation and results unverified.

0. What changed in versions 1.1 and 1.2

Changes in version 1.2

Version 1.2 applies an independent review of v1.1. The project, research questions and scope are unchanged.

#	Correction	Where
1	**B4-random and B2 are now part of the minimum result.** Without B4 the agent's contribution cannot be tested; B2 answers the agreed RQ2.	Section 7
2	**Call budget recomputed.** Two-stage configurations make two calls, so the development catalogue alone is 4,800 calls, not 3,600. The plan now gives an expected budget (about 21,000 calls) and a hard upper bound (about 36,700), and the exact test budget is computed at the freeze, before any test call.	Section 2A
3	**Caching cannot distort latency.** Cached outputs are used only for scoring. Latency comes from separate uncached timing runs. Cache keys cover the full rendered prompt, model digest and every generation setting.	Sections 2A, 5
4	**Determinism is checked, not assumed.** A repeat test on day 1 measures identical-output rate. Moving from Ollama to Transformers counts as a new environment and every compared system is rerun there.	Sections 2A, 5
5	**Prompt size is capped.** The training pool (10 per intent) is separate from what goes into one request: at most 16 examples and 2,000 input tokens.	Sections 2A, 4, 6
6	**B3 is described as the dataset's published reference baseline, not automatically SOTA.** Its OOS method (a confidence threshold tuned on development data) is now specified.	Section 3
7	**Unsupported extraction and runtime failures are primary metrics.** They directly support the verification claims.	Section 5
8	**The held-out project P3 is excluded from every pre-freeze step.** Gates G4–G10 use only P1 and P2.	Sections 4, 7
9	**Gate and screenshot renumbering map** from v1.0 to v1.2, so the tracker and evidence log can be synchronised.	Section 9
10	Table rows no longer split across pages in the PDF.	Layout

Changes in version 1.1

Version 1.1 applied the first plan review and added a zero-cost setup. Everything not listed in these two tables is unchanged from version 1.0.

#	Change	Where
1	New baseline **B4: plain search without the agent** (random and exhaustive search over the same catalogue, same budget). The catalogue is enlarged to 12 configurations so that choosing which candidates to try actually matters.	Sections 3, 6
2	Close prior work named: **Prompt2Model** and **DSPy**. The novelty claim must be positioned against both; DSPy is an optional search baseline.	Sections 1, 3, 13
3	**Data regime fixed:** all LLM systems use k = 10 training examples per intent. B3 is run twice: matched (same k) and full-data (upper reference).	Sections 3, 4
4	**Build-order bug fixed:** the shared execution catalogue is now built before the baselines. B0 is now in the final runs. Gates renumbered.	Section 7
5	**Compute plan added:** local open model via Ollama, a throughput test on day 1, prediction caching, and a budget of about 12,000 calls (recomputed in 1.2).	New section 2A
6	**MASSIVE only.** Projects are groups of MASSIVE domains; out-of-scope requests come from reserved domains. CLINC150 becomes an optional stretch goal.	Section 4
7	**Project descriptions frozen** before any development run, to close a leakage path.	Section 4
8	**Span-location rule** for turning LLM output into character offsets, tested in the evaluator gate.	Sections 5, 7

#	Change	Where
9	**Metrics split** into primary and secondary. One helper function creates every run folder.	Sections 5, 10
10	**DA1/DA2 status check first**, a minimum submittable result by 16 Oct, the paper skeleton from day 1, and an 18 Oct buffer day.	Sections 11, 12
11	**Provisional success thresholds** written now, to be confirmed before the freeze.	Section 5
12	**Zero-cost setup:** every tool, dataset and model is free. Free API tiers are limited to the Architect's planning calls.	New section 2A

1. What we are building

We are building a framework that accepts a project's description, supported intents, entity schema, example utterances and operating constraints. It analyses those requirements, proposes candidate NLP pipelines, evaluates them on development examples, selects an eligible pipeline and exports a versioned package that a backend can load.

The first implementation covers English, single-turn text requests, intent classification, entity extraction, schema validation and basic out-of-scope detection. The larger vision includes other NLP tasks, but those are future extensions.

The intended user is a developer who needs language understanding for a particular application. The decision supported is which pipeline configuration to deploy under a stated quality, latency and resource budget.

An example project might support order tracking and cancellation. Given "Cancel order 1234", its runtime should identify cancel_order and extract order_id=1234. This is an illustrative example, not a chosen benchmark or achieved result. The framework produces structured understanding; it does not actually cancel orders.

Two separate workflows

```
DESIGN TIME
Project description + schema + permitted train/development data + constraints
    -> Project Analyzer
    -> validated project specification
    -> NLP Architect
    -> bounded candidate generation (from a fixed 12-configuration catalogue)
    -> evaluation and verification on development data
    -> candidate selection, or an explicit no-eligible-candidate result
    -> export pipeline configuration, prompts, artifacts and manifest

RUNTIME
New request + frozen exported pipeline
    -> preprocessing
    -> intent/entity prediction and OOS decision
    -> schema and evidence checks
    -> structured result or explicit abstention/error
```

Design-time evaluation must never query final test labels. Runtime must load the exported package without asking the Architect to redesign it.

Proposed research contribution

The hypothesis is that project analysis plus bounded candidate generation and verification improves the quality or setup efficiency of project-specific NLP pipelines. We must test this hypothesis and compare it with existing work before claiming novelty. Calling a model "an agent" is not itself evidence of a contribution.

Closest prior work (new in 1.1). Two published systems are very near our claim, and the paper must position against both:

- **Prompt2Model** (Viswanathan et al., 2023) takes a natural-language task description and trains a small special-purpose model for deployment, using dataset retrieval, LLM-generated data and

fine-tuning.

- **DSPy** (Khattab et al., ICLR 2024) treats LM pipelines as programs and uses a compiler to optimise prompts and demonstrations against a metric. That is close to our "candidate generation plus evaluation-driven selection".

Our possible difference must be stated precisely and then tested. Candidate differences are: an explicit project specification with schema and constraints; a bounded, trusted component catalogue; constraint-aware selection with an explicit no-eligible result; evidence-checked entity output; and a frozen, versioned runtime package. If the agent adds nothing over plain search (baseline B4), the honest contribution is automation and setup effort, not search quality.

"Domain-adaptive" initially means adaptation through project specifications, examples and pipeline selection. Fine-tuning is optional. We will state the exact adaptation mechanism and avoid implying that model weights were adapted when they were not.

2. Where we do the work

Tool or location	Role in this project
Python project on the laptop	Main implementation and repeatable experiment commands
Ollama with a local open model	Runtime LLM for every LLM-based system (see section 2A)
Git and GitHub	Code history, configuration versions, documentation and team contributions
pytest	Automated component and integration checks
scikit-learn	Intent/OOS classification metrics, TF-IDF baseline and similarity retrieval
Project-specific scoring code	Exact entity matching, schema checks and supported extraction checks
JSON Schema validation	Validate input specifications and runtime output contracts
Jupyter notebooks	Inspect data and explain results; import the shared project code
PyTorch + Hugging Face Transformers	Fine-tuning the B3 encoder baseline on Kaggle
Kaggle notebooks (free GPU)	B3 fine-tuning; fallback host for the runtime model if the laptop is too slow
Saved JSON/CSV reports and charts	Permanent records of each experiment
Overleaf (free plan)	IEEE-format paper, started on day 1
draw.io	Architecture and module diagrams
Simple local demonstration interface	Show project configuration, selected pipeline and runtime requests

PyTorch is a machine-learning framework. Jupyter is a notebook environment. Neither automatically creates a complete research record. We will explicitly save run metadata and predictions, and keep the tracker and evidence index outside notebooks.

Planned repository layout

This is a proposed layout for the implementation repository; these folders have not been created by this planning task.

```
project/
  README.md
  pyproject.toml                 # dependencies and run instructions
  src/
    analyzer/                   # project-specification extraction
    architect/                  # plan and candidate configuration
    pipelines/                  # shared 12-configuration execution catalogue
    baselines/                  # B0 TF-IDF, B3 fine-tuned encoder, B4 search
    evaluation/                 # common scorer, span locator, experiment runner
    runtime/                    # package loading and inference interface
    llm/                        # one client wrapper, prediction cache, call counter
  tests/                        # automated checks and small fixtures
  configs/                      # project definitions, frozen descriptions, settings
  data/
    manifests/                  # source, license, split IDs and hashes
    processed/                  # data according to licensing/access rules
  cache/                        # cached LLM predictions keyed by config + case + model
  notebooks/
```

```
    01_data_review.ipynb
    02_development_results.ipynb
    03_final_results.ipynb
    kaggle_b3_finetune.ipynb
  runs/<run_id>/                  # immutable experiment output
  evidence/screenshots/          # milestone captures
  docs/
    PROJECT_PLAN.md
    PROGRESS_TRACKER.md
    EVIDENCE_LOG.md
    protocol.md
    decisions.md
    contributions.md
  reports/                       # submission documents and exported figures
```

The three planning files currently delivered under outputs become the source records until we choose the implementation repository. At setup, move or copy them into docs and explicitly identify the authoritative copies. Do not maintain two independently edited trackers.

2A. Zero-cost setup and compute budget (new in 1.1)

This is a college project, so the whole plan is designed to cost nothing. No paid API tier, paid model or cloud GPU rental is required.

What is used, and what it costs

Need	Free option	Limit to plan around
Code, tests, notebooks	Python, scikit-learn, pytest, Jupyter, PyTorch, Transformers	None
Version control	GitHub free account	None for this project
Datasets	MASSIVE (and optionally CLINC150), open licences with attribution	Confirm the licence in the G2 audit
Runtime LLM for B1, B2, B4, A1 and ablations	**Ollama** running a small open instruct model locally (about 3B–8B parameters, quantized)	Speed depends on the laptop; measured on day 1
B3 fine-tuning	**Kaggle notebooks** free GPU	Weekly GPU quota, commonly reported as about 30 hours; 12-hour sessions
Architect planning calls only	Free tier of Gemini or Groq	Daily request and token caps; data may be used by the provider
Paper	Overleaf free plan with the IEEE template	Collaborator limits on the free plan
Diagrams	draw.io	None

Why a local model instead of a free API

Free API tiers are too small for the full experiment. As an example, Groq's published free-plan sample limits for text models are about 1,000 requests a day and 200,000 tokens a day. A few-shot prompt is roughly 1,000–1,500 tokens, so that is only about 150 calls a day. Gemini's free tier is also rate-limited, and on that tier the provider may use inputs and outputs to improve its models.

A local open model has no rate caps or daily quota. Its weights are fixed, so results are reproducible, whereas free API models can change or be retired without notice. This reproducibility argument can be stated in the paper.

Rules for the models

- **One runtime model for every LLM-based system.** B1, B2, B4, A1 and all ablations use the same local model, the same quantization and temperature 0. This keeps the main comparison fair.
- **The Architect may use a free API model** for its planning calls, which are few (about 5–10 per project per design run, so about 100 in total). Record the provider, model ID and date for every call.
- Free API models are never used for runtime predictions in reported results.

- Send only public dataset content to free APIs; never send private data.
- Record the exact model name, tag/digest, quantization and Ollama version in every run manifest.
- **Determinism is checked, not assumed.** Fixed weights and temperature 0 do not guarantee identical outputs across hardware, library versions or batching. The day-1 test measures it, and the result is reported.
- **One environment per comparison.** If the runtime moves from Ollama on the laptop to Transformers on Kaggle, that is a changed environment, not the same model. Every system in a reported comparison must then be rerun in the new environment; results from the two environments are never mixed in one table.

Day-1 throughput test (part of G0)

1. Install Ollama and pull two candidate models: one about 3B and one about 7–8B.
2. Run 50 representative few-shot requests through each, with JSON output. Use realistic prompts at the full size allowed by the prompt cap below (about 2,000 input tokens), not short toy prompts.
3. Record median and p95 seconds per request, input and output tokens, and the JSON-validity rate.
4. **Determinism check:** run the same 50 requests a second time with caching disabled and record the identical-output rate. If it is below 100%, report it as a limitation; the cached first output is then the recorded prediction for every system, so all systems are scored on one consistent set of outputs.
5. Pick the model by this rule: the largest model whose estimated run time for the *expected* budget (below) fits in about two overnight batches and whose JSON validity on the 50 requests is at least 95%.
6. If even the 3B model is too slow, move to a Kaggle GPU using Transformers. Under the one-environment rule above, every compared system is then run there.

Prompt-size cap

The k = 10 examples per intent form the *training pool*; they are not all put into one prompt. Each request includes **at most 16 examples and at most 2,000 input tokens**, whichever limit is reached first. Fixed selection includes one example per intent, up to that cap; similarity selection includes the nearest examples, up to that cap. If the cap would be exceeded, examples are dropped first; intent and entity definitions are never truncated. Actual token counts are logged per request.

Prediction caching (makes the budget feasible)

The runtime model's weights and settings are fixed, and every candidate is a configuration from the fixed catalogue. So each prediction is computed once, cached in `cache/`, and that recorded output is used by every system that needs it (the determinism check above reports how stable a repeat would have been). A1's repeated design runs, B4's random-search seeds, exhaustive search and several ablations then reuse cached predictions instead of calling the model again.

Design-overhead metrics must still report *logical* calls (what the method would have made on its own) separately from *physical* calls (what was actually executed). Cache hits are not counted as free design cost.

Rules that keep the cache honest:

- **Cache key** = hash of the full rendered prompt (instructions, definitions and the exact examples included), the model name and digest, and every generation setting (temperature, seed, maximum tokens, output format, stop sequences). Two prompts that differ in any character are different entries.
- **Cached outputs are used for scoring only, never for latency.** Every cache hit is flagged in predictions.jsonl.
- **Latency comes from separate timing runs:** for each system and project, 50 fixed development cases run with the cache disabled, after a warm-up of 5 requests. These timings are the only source for latency figures and the L_max check.
- A JSON-repair retry is a new call with its own cache entry and is counted in the budget.

Estimated call budget (recomputed in 1.2)

Assumptions: 3 projects; per project, 100 development cases and 300 test cases; 12 catalogue configurations, of which 4 are two-stage (two calls per request) and 8 are single-call. A 10% allowance covers JSON-repair retries.

Block	Calculation	Expected calls	Upper bound
Development predictions, full catalogue (covers A1, B4 random and exhaustive, no-selection and fixed-candidate ablations)	(8 × 1 + 4 × 2) calls × 100 dev × 3 projects	4,800	4,800
No-Analyzer ablation: its own 4-candidate search with different prompt content	4 configs × 100 × 3, about 1.33 calls each; bound assumes all two-stage	1,600	2,400
B2 manual development checks (hand-written descriptions, so new cache entries)	about 4 configs × 100 × 3	1,600	2,400
Final test: distinct configurations selected by all frozen systems, including all Architect repeats and B4 seeds	expected about 8 configs × 1.33 × 900; bound = full catalogue on test (14,400) + up to 3 no-Analyzer configs (5,400) + B2 (1,800)	9,600	21,600
Uncached latency timing runs	50 cases × about 6 systems × 3 projects × 1.33, plus warm-ups	1,300	2,000
Day-1 throughput and determinism test	2 models × 50 × 2 runs	200	200
Retries	about 10%	1,900	3,300
Total		**about 21,000**	**about 36,700**

What this means in time. At 3 seconds per call, the expected budget is about 17–18 hours (two overnight batches); at 6 seconds it is about 35 hours, which does not fit. The day-1 test decides which case applies.

The test budget becomes exact at the freeze. Selection happens on development data, so by G10 every system's final configuration is known. Before any test call, count the distinct (configuration, prompt content) pairs to be run on test, multiply by test cases and calls per request, and record the exact number in protocol.md. If it does not fit the remaining time, reduce in this order before running anything: test cases from 300 to 200 per project; B4 seeds from 5 to 3; Architect repeats reported on test from 3 to 1 (the first, decided before the freeze). Never reduce by dropping failed cases or selectively re-running.

B0 and B3 make no LLM calls.

3. Research questions and fair comparisons

RQ1: Does an agent-generated pipeline improve intent and entity performance over a fixed generic pipeline on the same projects?

RQ2: How does it compare with a manually designed project-specific pipeline, considering quality, configuration effort, design cost and runtime latency?

RQ3: What do project analysis, multiple candidates and evaluation-driven selection contribute individually?

RQ4: Does the frozen framework work on a project held out from framework development?

RQ5 (new in 1.1): Does the agent choose candidates better than plain search with the same budget?

Required comparison systems

ID	System	Comparison rule
B0	Simple reference: TF-IDF intent classifier with a documented OOS threshold	Sanity reference, not automatically SOTA; included in final runs
B1	Generic LLM pipeline with a fixed prompt and procedure	Same local model; receives supported labels/schema and the same permitted information; no project-specific search
B2	Manually configured project-specific pipeline	Same local model and same catalogue; human picks a configuration and hand-writes intent/entity descriptions; 2-hour budget per project, recorded

ID	System	Comparison rule
B3	Fine-tuned encoder following the MASSIVE paper's published reference baseline (XLM-R family)	Run twice: **B3-k** with the same k = 10 examples per intent, and **B3-full** with the full training split as an upper reference. Trained on in-scope intents only; OOS by a maximum-softmax confidence threshold tuned on development data, the same rule type as B0
B4	Plain search without the agent	Same catalogue, same Analyzer output, same scorer. **B4-random**: 4 configurations drawn at random (5 seeds). **B4-exhaustive**: all 12 configurations, as a development-score ceiling
B5	Optional: DSPy optimiser over the same task	Only if time allows after the minimum result; same local model and examples
A1	Our full agent-generated pipeline	Same input information, evaluation cases and declared resource constraints; 4-candidate budget

B1, B2, B4 and A1 are the central framework comparison. B3 is our **published reference baseline**: the MASSIVE paper reports XLM-R and mT5 modelling results with released code and models, so B3 has a published, reproducible basis. Publication alone does not make it current state of the art. In G5, confirm the exact model and training recipe from the official repository, and check the DA1 literature review for any stronger, newer method on MASSIVE. Describe B3 in the paper as "the dataset's published reference baseline" unless that check supports a SOTA claim. The same caution applies to every system: a generic LLM or simple classifier must not be described as SOTA without supporting evidence.

How B3 handles OOS. The fine-tuned encoder is trained only on the project's in-scope intents, never on OOS-pool examples (no LLM system sees OOS examples either). At inference, a request is labelled OOS when its maximum softmax probability is below a threshold tuned on the project's development set. The same tuning procedure is used for B0's TF-IDF threshold. Slots are predicted as BIO tags and converted to character spans with the same evaluator.

Why B4 matters. A1 selects among candidates using development scores; B1 gets no search at all. If A1 beats B1, the simplest explanation is "tuning on development data helps", not "the agent helps". B4 separates the two. A1 should be compared with B4-random at an equal budget. B4-exhaustive shows how much quality is left on the table by any 4-candidate budget.

Data regime. All LLM systems (B1, B2, B4, A1) receive k = 10 training examples per intent. B3-k is the fair supervised comparison. B3-full is reported as an upper reference and must not be presented as a matched comparison. A fine-tuned encoder with full data may well outperform every few-shot system; if so, report it plainly. The likely honest framing for A1 is the low-data and setup-effort regime.

Control the base model, data access, allowed examples and output schema for the principal comparison. Record search budgets and human effort. If models or budgets differ, identify that comparison as a combined-system comparison rather than attributing all gains to the agent design. For B2, record any AI assistance.

Published numbers are directly comparable only when dataset version, split, labels and scoring match. Our project subsets require rerunning baselines; their scores must not be placed beside full-benchmark published scores as though the tasks were identical.

4. Data plan and protection against leakage

Dataset: MASSIVE only (changed in 1.1)

MASSIVE contains 1M parallel, labelled virtual-assistant utterances across 51 languages, 18 domains, 60 intents and 55 slot types. We use only the English (en-US) portion. It provides intent and slot annotations, so one dataset and one span-conversion path cover intent, entities and project-relative out-of-scope detection.

CLINC150 is now an optional stretch goal only. It has no entity annotations, so it would need a separate intent/OOS pipeline and adds work without strengthening the main comparison.

At the data milestone (G2), record version, licence, language, actual counts, label inventory, split availability and annotation format.

Projects from MASSIVE domains

Projects are groups of MASSIVE domains with a coherent purpose. A proposed starting point, to be confirmed in G2 by domain semantics before any score is computed:

Project	Role	MASSIVE domains (proposed)
P1 Home assistant	Development	alarm, iot
P2 Productivity	Development	calendar, email, lists
P3 Ordering and travel	Held-out	takeaway, transport
OOS pool	Out-of-scope source for all projects	domains used by no project, for example cooking, news, weather, social, qa

Out-of-scope requests for every project are drawn from the reserved OOS pool, never from another project's domains. This keeps projects independent and avoids leaking the held-out project's domains into development.

Split sizes

Split	Per project	Source
Train (example pool)	k = 10 examples per intent; at most 16 examples and 2,000 input tokens per request (see prompt-size cap)	Official train split, fixed sampling seed
Development	100 cases: about 80 in-scope + 20 OOS	Official dev split, stratified, fixed IDs
Test	300 cases: about 250 in-scope + 50 OOS (fallback 200)	Official test split, stratified, fixed IDs

Preserve official split assignments. Report that these are subsets and list their IDs in the manifests.

Three kinds of evidence

1. Standard benchmark evaluation where practical: preserves the dataset's task and scoring rules.
2. Project adaptation evaluation: documented project schemas/domains drawn from MASSIVE, with a fixed mapping of supported intents and entity types.
3. Challenge cases: human-reviewed examples covering missing values, ambiguous requests, malformed input and unsupported requests. Label these as diagnostic tests, not a replacement for the benchmark.

Aim for two development projects and one held-out project. With only one held-out project, state that the generalisation evidence is limited.

Frozen project descriptions (new in 1.1)

The Analyzer's input is a free-text project description that we write ourselves. If it is written after looking at development errors, it can encode answers. Therefore:

- Write all three project descriptions, including the held-out project's, in G2, before any development run.
- Keep them short and realistic: what a developer would plausibly write (purpose, users, supported requests, constraints), not a list of tricky cases.
- Commit them to `configs/` and record the commit hash in protocol.md. Any later edit counts as a protocol change and must be logged.
- Report in the paper that descriptions were author-written; this is a limitation.

Split and access policy

- Training data: fitting models or selecting permitted few-shot examples.
- Development data: choosing prompts, candidates, thresholds and hyperparameters.
- Final test data: used only after framework, configurations, metrics and budgets are frozen.

- Held-out project: framework design does not use this project's outcomes. **Before the freeze, P3 is excluded from every step except writing and committing its description in G2:** no development predictions, no threshold tuning, no prompt checks and no smoke tests on P3 data. Every build-gate experiment (G4–G10) uses only P1 and P2. After freeze, adaptation may access P3's specification and allowed train/development data; its test set stays hidden until the final evaluation. This measures adaptation to a new project, not necessarily zero-shot learning.
- Candidate-building tools receive only train/development file paths. The independent final evaluator has test access.
- Check exact and near-duplicate overlap across splits; report the rules.
- Synthetic development examples must be clearly marked. Do not use synthetic labels from the same generator as sole final ground truth.
- Tune OOS thresholds only on development data with documented in-scope and OOS composition.
- Do not repeatedly tune after examining final test results. If that happens, mark the test as used for development and require a new untouched confirmation set.
- Record possible public-data familiarity of pretrained models as a limitation; a locally hidden test file does not eliminate pretraining contamination. Note also that data sent to free API tiers may be retained by the provider.

5. What we measure

Primary metrics (reported for every system and project)

Metric	Definition and reporting rule
Intent macro-F1	Equal-weight mean of per-intent F1 on in-scope requests; an OOS rejection of an in-scope case is an error
Entity span F1	Exact entity type and character-span match, using verified conversion from MASSIVE annotations; report micro precision/recall/F1
OOS F1 and false rejection	Treat OOS as positive; report precision, recall, F1 and the false-rejection rate of in-scope requests
Schema validity	Valid responses divided by all evaluated requests, before and after repair; timeouts, unparseable output and missing responses fail
Unsupported extraction rate	Unsupported predicted entity items divided by all predicted entity items; publish numerator and denominator; N/A when nothing is predicted
Runtime failures	Error and timeout rate, retry count and fallback rate; failed requests stay in every denominator
Runtime latency	Median and p95 wall-clock time per request from the uncached timing runs only, including validation/retries; cold-start measured separately
Design cost	Logical and physical model calls, tokens, and wall time for design; cost is 0 on the free setup but calls and time are still reported

Secondary metrics (reported where time allows)

Metric	Definition and reporting rule
End-to-end exact match	Fraction with correct intent/OOS decision, complete correct entity set and valid schema
Entity value accuracy	Canonicalised values; define permitted normalisation before evaluation
Unsupported request rate	Requests with at least one unsupported entity divided by all evaluated requests
Setup effort	Timed human configuration activity for B2 and A1; distinguish compute time, waiting and active human time

Caveat on setup effort. The same person builds A1 and does the B2 manual configuration, and already knows the data. Report the timing, but state this bias as a limitation; it is a single, non-blind measurement.

Span-location rule (new in 1.1)

LLMs return entity strings, not character offsets. The evaluator converts them deterministically:

1. Search for the exact returned string in the original request text (case-sensitive first, then case-insensitive).
2. If it occurs more than once, take the first occurrence not already assigned to another predicted entity, scanning left to right.

3. If it is not found, the entity is **unsupported**: it counts as a false positive for span F1 and in the unsupported-extraction numerator. Never fuzzy-match it into a span.
4. Strip only leading/trailing whitespace and surrounding quotes before matching. No other normalisation.

This rule is implemented once in `src/evaluation`, used by every LLM system, and covered by unit tests in G3 (repeated substring, missing substring, case difference, overlapping entities).

For extractive entities, output text evidence and character offsets. A wrong entity type can be supported by the text but still be semantically wrong; entity F1 catches that error. Missing gold entities are recall errors, not unsupported extractions.

If there are zero predicted entities, unsupported extraction rate is N/A, and entity recall remains visible. Empty outputs must not appear to be a successful solution. Validation must not silently invent missing values.

Score malformed responses and timeouts explicitly. Never remove unsuccessful requests from the denominator.

Success rules to freeze before the final test

The primary decision compares A1 with B1 on per-project intent macro-F1 and entity F1. The agent-specific claim (RQ5) compares A1 with B4-random at equal budget.

Provisional thresholds (new in 1.1). Confirm or adjust these in protocol.md by 16 October, before any test access, and never after:

Symbol	Meaning	Provisional value
delta_I	Minimum meaningful intent macro-F1 improvement of A1 over B1	+2.0 points
epsilon_E	Largest acceptable entity F1 drop of A1 relative to B1	2.0 points
S_min	Minimum schema validity after repair	0.98
L_max	Maximum p95 latency for A1	2 × B1's p95 on development data, fixed in protocol.md as a number

Rule: "A1 succeeds against B1 if intent improvement ≥ delta_I, entity difference ≥ −epsilon_E, schema validity ≥ S_min and p95 latency ≤ L_max." Comparison with B2 may demonstrate proximity in quality plus lower setup effort rather than superior accuracy.

Repeat the Architect's design run three times per project (its free API model can be non-deterministic). Runtime predictions are computed once per (configuration, case) and cached; the day-1 determinism check reports how often a repeat would have produced a different output. Report that rate rather than assuming identical outputs or inventing variance. Use paired bootstrap confidence intervals for score differences on the same test examples and report per-project results. With three projects, make no cross-project generalisation claim. These intervals describe uncertainty within the evaluated data, not certainty over all future projects.

6. Architecture implementation decisions

Project Analyzer

Input: frozen project description, supported schema, labelled examples and constraints. Output: validated ProjectSpec with intent definitions, entity types, required/optional fields and constraint values. Authoritative supplied schema fields take precedence over inferred fields. Flag contradictions or missing information rather than filling in unsupported assumptions.

Tests: compare with manually checked specifications; reject duplicate/conflicting labels; handle absent optional fields; preserve constraints; ensure held-out test paths are not accessible through the data interface.

Execution catalogue (changed in 1.1)

The catalogue is a fixed set of 12 declarative configurations, built from three factors:

Factor	Options
Extraction strategy	(a) joint single call: intent + entities together; (b) two-stage: intent first, then entities for that intent's slots; (c) schema-guided JSON with field descriptions
Example selection	(a) fixed: one example per intent; (b) similarity-retrieved examples (TF-IDF nearest neighbours from the k-shot pool). Both respect the cap of 16 examples and 2,000 input tokens
OOS policy	(a) LLM "none of the above" label; (b) TF-IDF similarity threshold tuned on development data

$3 \times 2 \times 2 = 12$ configurations. All are built as trusted Python components; no generated code runs. B1, B2, B4 and A1 all execute through this same catalogue. B1 is fixed to one predeclared configuration (joint single call, fixed examples, LLM label).

NLP Architect and candidate builder

The Architect reads the ProjectSpec and chooses which catalogue configurations to evaluate. It does not need unrestricted generated Python code.

Search budget: **4 candidate evaluations per project** (3 initial + 1 revision chosen after reading the development error report). With 12 configurations and a budget of 4, the choice of which candidates to try is a real decision, which is what B4 tests. Save every attempted configuration, its rationale and its failures. If the budget is exhausted or no candidate meets hard constraints, return that outcome explicitly.

Evaluator and selector

Run the common development scorer for each candidate. Filter by declared hard constraints. Use a fixed ranking rule: highest intent macro-F1 among candidates meeting the entity and latency floors, with tie-breaks by entity F1 and then lower p95 latency. Do not hide trade-offs inside an unexplained weighted average.

Runtime and export

Export the ProjectSpec, chosen configuration, prompts, model identifiers (Ollama model name and digest), dependency manifest and version/hash. Provide a simple callable interface first; add an HTTP wrapper and small demo only after correctness tests pass.

Output fields should distinguish status, intent, entities with evidence, schema validity, pipeline version and timing. Use explicit OOS/abstention/error statuses. Include confidence only if its source and meaning are defined; a model's self-reported number must not be presented as calibrated probability.

7. Build order with tests and evidence (reordered in 1.1)

Each gate has build work, a test and an evidence checkpoint. Automated test results are required to mark a technical gate verified; screenshots are supporting documentation. The shared execution catalogue (G4) now comes before the baselines that depend on it. **Gates G4–G10 use only the development projects P1 and P2.**

Gate	Build and deliverable	Test / completion condition	Evidence and screenshot
G0 Requirements	Confirm DA1/DA2 status with the teacher; recover existing work; install Ollama; run the 50-request throughput test; choose the runtime model; start the Overleaf IEEE skeleton	Every unresolved decision listed; model chosen by the section 2A rule; no unverified work marked done	Requirement matrix, throughput report, environment report; S00
G1 Setup	Repository structure, dependency lock, LLM client wrapper with cache and call counter, run-folder helper	Fresh environment runs a small test; cache hit/miss counted correctly; no secrets in logs	Commit, setup instructions, smoke-test report; S01
G2 Data	Download MASSIVE en-US; define P1–P3 and the OOS pool; sample k-shot pools and dev/test IDs; write and commit the frozen project descriptions	Label/schema integrity, span conversion and leakage checks pass; descriptions committed before any LLM run	Data manifest, counts, description commit hash; S02
G3 Evaluator	Scoring, span-location rule, latency and failure handling	Hand-computed perfect/wrong/empty/malformed fixtures and all span-rule cases match expected results	Unit-test report and metric definitions; S03

Gate	Build and deliverable	Test / completion condition	Evidence and screenshot
G4 Catalogue + B0/B1	Build the 12-configuration execution catalogue; B0 TF-IDF; B1 fixed configuration; start B3 Kaggle training (P1, P2 only)	All 12 configurations run on 10 P1/P2 dev cases; B0 and B1 scored on P1/P2 development data	Catalogue tests, B0/B1 development outputs; S04
G5 B2/B3/B4	B2 manual configuration (timed); B3-k and B3-full trained and scored; B4 random/exhaustive search code	B3 reproduces a sensible score on the official split; B4 selects deterministically per seed	Baseline configs, timings, per-example development outputs; S05
G6 Analyzer	Convert project inputs into validated ProjectSpec	Matches gold specification fixtures; conflicting input is flagged	Specification comparisons and tests; S06
G7 Architect	Choose candidate configurations within the 4-evaluation budget	Choices are valid catalogue entries; budget cap respected; rationale logged	Candidate plans and tests; S07
G8 Selection	Evaluate candidates and choose under declared rules; fill the P1/P2 development cache overnight	Known synthetic score tables select the correct candidate; no-eligible case handled	Development leaderboard and decision log; S08
G9 Runtime	Export/reload chosen pipeline; request interface and simple demo	Fresh process loads artifact; normal/OOS/invalid requests work; failures explicit	Integration tests, exported manifest and demo; S09
G10 Freeze	Diagnose development errors; fix budgets, ablations and thresholds; compute and record the exact test call budget; **minimum submittable result check**	Test answers still unopened; P3 data still untouched; protocol, thresholds, configurations and code frozen in one commit	protocol.md and freeze commit; S10
G11 Final evaluation	Run frozen B0/B1/B2/B3/B4/A1 and ablations on P1 and P2; then run frozen design (A1, B4, B2, B3) on P3 development data and evaluate everything on P3 test; uncached timing runs	All runs have matching manifests, complete predictions and scored failures	Immutable runs, comparison tables, CIs and plots; S11
G12 Reproduction + submission	Recreate environment, rerun one experiment, finish paper and figures	Metrics recompute from saved predictions; every quantitative claim traces to a run	Reproduction log, final documents, demo rehearsal; S12

If a gate fails, log the failure, fix it and record a new test result. Do not delete the original failure record or replace a screenshot while keeping the old run ID.

Minimum submittable result (new in 1.1)

If time runs short, this is the smallest set of results that still supports an honest paper: **B0, B1, B2, B3-k, B4-random and A1 on P1 and P2**, plus the no-evaluation-driven-selection ablation, all through the frozen protocol, with uncached latency and all failures recorded. B4-random is essential because without it the agent's contribution (RQ5) cannot be tested; B2 is essential because it answers the agreed RQ2. Freeze at the end of 16 October with whatever meets this bar.

Everything else is added in this order if time allows: the held-out project P3; the remaining ablations; B3-full; B4-exhaustive; DSPy (B5); CLINC150.

8. Ablations and error analysis

Run the full system and these controlled variants on the same splits:

- **No free-text Project Analyzer:** directly use the supplied authoritative structured schema; keep all other information available. This isolates inferred enrichment without depriving the baseline of required labels.
- **One fixed candidate:** removes candidate search while preserving the runtime components.
- **No evaluation-driven selection:** generate the same candidate pool and choose by a fixed pre-registered rule rather than development scores.
- **No post-validation/repair:** compare raw outputs with the full validation path; retain the same scorer and record invalid output honestly. This needs no extra calls, because raw outputs are logged before repair.

Most ablations reuse cached catalogue predictions. Report which metrics change and the compute saved or added. A component that helps schema validity but hurts latency is a trade-off, not an unconditional improvement.

Classify errors: confused intents, missed entities, wrong boundaries, wrong types, unsupported values, incorrect OOS rejection, missed OOS, invalid schema and runtime failures. Inspect development examples during improvement. Inspect final-test errors only after freeze and label further fixes as later work requiring fresh confirmation.

9. How we track progress and capture proof

PROGRESS_TRACKER.md is the main file to check before each work session. It names the current gate, next task, owner, dependencies, test and evidence needed. EVIDENCE_LOG.md is the index of tests, screenshots, runs and decisions. PROJECT_PLAN.md explains the agreed approach.

At the start of a session: read the tracker, state the target gate and identify any needed input. Before a milestone test: tell the user what will be checked and which screenshot ID applies. After the test: save machine-readable reports, update status and link the evidence. At session end: record completed work, failures, next action and any user input needed.

Notification wording to use during implementation: "Testing checkpoint S06: checking whether the Analyzer preserves the project's intents and required entities. I will save the test report; this is a useful screenshot point." Do not claim a checkpoint passed before the report exists.

Screenshot captures should show the test/report title, run ID, date and visible result. Capture data summaries, component tests, candidate comparisons, runtime demonstration, final tables and the confusion matrix. Prefer an exported chart for paper figures rather than a screenshot of a chart.

The user may capture screenshots manually. The assistant may capture supported browser/tool views when those controls are available. Automatic capture of arbitrary desktop windows is not assumed. If a capture is unavailable, save the report and mark the screenshot Pending; do not block valid experiments merely to obtain a picture.

Suggested screenshot filename: S06_20261014_analyzer-check_RUN-ID.png. Store each in the repository evidence/screenshots folder and register its exact path. Screenshots are supplementary; the saved inputs, outputs, metrics and code version are the reproducible evidence.

The tracker is updated during active project work. It is not an unattended monitor and will not automatically change while no session is running.

Gate and screenshot renumbering (v1.0 to v1.2)

Version 1.1 inserted a catalogue gate and merged reproduction with submission, so gate and screenshot numbers changed. **Before starting work, update PROGRESS_TRACKER.md and EVIDENCE_LOG.md to the v1.2 numbers using this map**, and note the change in decisions.md. Any evidence already captured under a v1.0 number keeps its file name and gets the mapped v1.2 ID added in the log.

v1.0 gate / screenshot	v1.2 gate / screenshot
G0 Requirements / S00	G0 Requirements / S00 (now includes throughput and determinism test)
G1 Setup / S01	G1 Setup / S01
G2 Data / S02	G2 Data / S02
G3 Evaluator / S03	G3 Evaluator / S03 (now includes span rule)
G4 Baselines / S04	Split: G4 Catalogue + B0/B1 / S04, and G5 B2/B3/B4 / S05
G5 Analyzer / S05	G6 Analyzer / S06
G6 Architect / S06	G7 Architect / S07
G7 Selection / S07	G8 Selection / S08
G8 Runtime / S08	G9 Runtime / S09
G9 Development freeze / S09	G10 Freeze / S10
G10 Final evaluation / S10	G11 Final evaluation / S11
G11 Reproduction / S11 and G12 Submission / S12	G12 Reproduction + submission / S12

10. Experiment records

Every real run gets a unique ID such as YYYYMMDD_HHMM_project_system_seed. Never reuse an ID. A run folder should contain:

```
config.json          # system, project, model, settings and budgets
manifest.json        # code commit, environment, model digest, data/split hashes, times
predictions.jsonl    # case IDs, outputs, status, cache hit/miss, tokens, errors
metrics.json         # scores, sample counts, definitions/version and denominators
timing.jsonl         # uncached timing-run latencies (the only latency source)
```

```
events.jsonl          # candidate attempts, failures, retries and selection decisions
summary.md            # plain-language result and limitations
```

One helper creates every run folder (new in 1.1). A single function, for example `new_run(project, system, seed, config)`, generates the run ID, creates the folder, writes config.json and manifest.json (git commit, Python and package versions, Ollama model name and digest, data hashes), and returns a logger for predictions and events. No run is written by hand, so recording costs nothing per run.

During development, store per-example gold comparisons only for permitted train/development examples. After final evaluation, the independent evaluator may export final gold comparisons for the frozen reporting stage. Keep credentials and unnecessary private information out of logs and screenshots.

Maintain a combined experiment table derived from metrics files. Never type attractive scores into charts manually. Costs that cannot be measured are marked Unknown, not zero. Failed and incomplete runs remain visible.

11. Schedule and priorities (revised in 1.1)

The supplied handout lists DA1 on 7 August 2026, DA2 on 20 September 2026 and DA3 on 20 October 2026. As of 11 October, the first two dates are past and DA3 is nine days away.

First action, before coding: confirm with the teacher whether DA1 and DA2 were submitted and accepted. DA2 required implementation and a published/SOTA comparison; if it is still owed, priorities change and this schedule must be adjusted. Also confirm the actual DA3 cutoff time.

The following is a conditional compressed plan, not a completion guarantee.

Date	Target	Priority
11 Oct	G0–G1: DA status check; Ollama throughput test and model choice; repository, cache and run helper; Overleaf IEEE skeleton with section headings	Essential
12 Oct	G2–G3: MASSIVE projects, splits and manifests; frozen project descriptions; evaluator with span rule; draft related work (Prompt2Model, DSPy, the 15 DA1 papers)	Essential
13 Oct	G4: 12-configuration catalogue; B0 and B1 on P1/P2 development; start B3 training on Kaggle	Essential
14 Oct	G5–G6: B2 manual configuration (timed), B3-k scored, B4 search code, Analyzer	Essential
15 Oct	G7–G8: Architect and selection; fill the development cache overnight	Essential
16 Oct	G9–G10: runtime export; minimum-result check; exact test budget recorded; protocol, thresholds and code freeze. Start P1/P2 test runs overnight	Essential
17 Oct	G11: P1/P2 final evaluation, uncached timing runs, ablations, bootstrap CIs	Essential
18 Oct	**Buffer:** reruns for failures; held-out project P3 (design on its development data, then test); B3-full if not done	Buffer
19 Oct	G12: reproduction; results section, figures and limitations	Essential
20 Oct	Final review and submission before the confirmed cutoff	Essential

Write the paper's introduction, related work and method as implementation proceeds; keep the results section empty until measurements exist. If time is insufficient, reduce in this order: interface polish, CLINC150, DSPy (B5), B4-exhaustive, B3-full, extra ablations, held-out project P3. B2 and B4-random are not on this list; they belong to the minimum result. Never reduce the integrity of data splits, baseline fairness or evidence records. Any unmet published-baseline or submission requirement remains explicitly incomplete.

12. Assignment alignment and final package

DA1: quantified domain motivation and stakeholder; at least 15 papers with the required recency/venue distribution; comparison table; 3–5 derived gaps; a one-paragraph input/output/constraint/success statement; full pipeline and detailed module diagrams; citations or hypotheses for choices; dataset/split/baseline/hardware/ablation plan; feasibility note; report PDF; separate high-resolution architecture; GitHub with dataset-loading code; contribution matrix. The 15-paper list should now include Prompt2Model and DSPy.

DA2: implementation, repeatable evaluation, published-baseline comparison (B3, following the MASSIVE paper's reference baseline; SOTA status checked against the literature, not assumed), results and contribution record. Detailed DA2 rubric is not supplied in the current file and should be checked if another handout exists.

DA3: IEEE-format paper or patent draft with result analysis. Our proposed route is an IEEE-format research paper, written on Overleaf. Publication is not mandatory in the supplied handout. Detailed formatting/submission instructions beyond the supplied overview remain to be confirmed.

The handout is inconsistent on DA1 length (4–5 suggested pages versus 8–12 in the checklist), conference-reference eligibility (reputed conferences listed but conference papers then limited), and team size (2–3 versus 1–3). Record the teacher's clarification instead of guessing. These administrative questions need not stop isolated technical development.

Final evidence package: source repository and setup instructions; data source/split manifests; frozen project descriptions; frozen experiment protocol; pipeline artifacts; baseline/agent/search/ablation runs; results with uncertainty and limitations; architecture and result figures; demonstration; contribution matrix; required report/paper. A GUI alone does not satisfy the research evidence requirement.

Final claim check: describe only improvements demonstrated by measured results. If the system improves schema validity or setup effort but not F1, state that precisely. If A1 does not beat B4 at equal budget, say the agent's choice adds no measurable search benefit and report the contribution as automation. If it fails to improve, report the negative result and identified limitations.

13. Sources and verification boundaries

- Teacher's supplied handout: D:/study/nlp/AI DIGITAL ASSIGNMENT_1.docx, read in an earlier conversation. It gives DA1 details and the DA1–DA3 overview.
- MASSIVE paper (FitzGerald et al., ACL 2023): https://arxiv.org/abs/2204.08582 — 51 languages, 18 domains, 60 intents, 55 slot types; reports XLM-R and mT5 baselines with released code and models.
- MASSIVE official repository: https://github.com/alexa/massive — data and baseline implementation. Licence and actual local counts still need the G2 audit.
- Prompt2Model (Viswanathan et al., 2023): https://arxiv.org/abs/2308.12261 — closest prior work on description-to-deployable-model systems.
- DSPy (Khattab et al., ICLR 2024): https://arxiv.org/abs/2310.03714 — metric-driven optimisation of LM pipelines; optional baseline B5.
- CLINC dataset paper: https://aclanthology.org/D19-1131/ — intent and OOS evaluation; optional stretch goal only.
- Ollama: https://ollama.com — local open-model runtime.
- Groq rate limits: https://console.groq.com/docs/rate-limits — free-plan limits; the console limits page is authoritative for an account.
- Gemini API rate limits and terms: https://ai.google.dev/gemini-api/docs/rate-limits and https://ai.google.dev/gemini-api/terms — free-tier limits and data-use terms.
- Kaggle notebooks documentation: https://www.kaggle.com/docs/notebooks — session limits; the weekly GPU quota is shown in the account settings.
- scikit-learn classification report: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html — standard precision/recall/F1 reporting.
- pytest documentation: https://docs.pytest.org/en/stable/contents.html — automated testing.
- PyTorch workflow: https://docs.pytorch.org/tutorials/beginner/basics/intro — model workflow for the B3 baseline.
- PyTorch reproducibility notes: https://docs.pytorch.org/docs/stable/notes/randomness.html — why identical results are not guaranteed across platforms and versions.

Free-tier limits and model availability change frequently. The figures in section 2A were checked on 11 October 2026 and must be re-checked in the account consoles before relying on them.

The exact original 15 papers, five gap statements, existing presentation, study guides and implementation have not been inspected. The project title and agreed scope come from the user's supplied context. This plan preserves that scope but does not certify prior artifacts or create experiment results.